# Лекция 7 Сервис предсказаний и контракт API

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР7 «Сервис предсказаний и контракт API»

## Цель лекции

Показать API как устойчивую границу между моделью и клиентом. После занятия студент должен уметь проектировать строгий request/response contract, объяснять OpenAPI, readiness, status codes и request ID, оценивать совместимость изменений и связывать контракт с автоматическими тестами.

## План занятия

| Время | Содержание | Действие преподавателя |
|---|---|---|
| 0–10 мин | Модель и сервис — разные продукты | Разобрать путь запроса от клиента к артефакту модели |
| 10–25 мин | HTTP-контракт и момент прогноза | Вывести минимальный request из data contract |
| 25–40 мин | Pydantic и строгая валидация | Показать ошибки неявного приведения и лишних полей |
| 40–52 мин | Response и ошибки | Сопоставить 200, 422, 500 и 503 |
| 52–65 мин | OpenAPI и совместимость | Прочитать сгенерированную JSON Schema |
| 65–76 мин | Lifespan, health и readiness | Смоделировать процесс без загруженной модели |
| 76–84 мин | Наблюдаемость и request ID | Связать запрос, ответ, лог и метрику |
| 84–90 мин | Contract tests и ЛР7 | Сформулировать доказательства индивидуальной сдачи |

## 1 От модели к обслуживаемому решению

Начните с вопроса: «Что должен знать клиент, чтобы использовать модель?» Клиенту не нужен объект scikit-learn, путь к `joblib` или порядок колонок внутри DataFrame. Ему нужен стабильный договор: адрес, метод, допустимый JSON, ответ, ошибки и обещания по готовности.

Модель отвечает на вопрос о вычислении. Сервис дополнительно решает:

- когда и как загрузить модель;
- какие входы допустимы в момент прогноза;
- как преобразовать вход в признаки;
- как обозначить неготовность;
- как ограничить и документировать выход;
- как найти конкретный запрос в диагностике;
- как изменить контракт, не разрушив клиентов.

Поэтому «обернуть модель в FastAPI» недостаточно. Сначала проектируют внешний контракт, затем связывают его с реализацией.

## 2 Граница доверия

HTTP-вход — недоверенные данные. Клиент может прислать пропущенное поле, строку вместо числа, дополнительный ключ, чрезмерно длинный идентификатор или признак, который существует в датасете, но неизвестен на момент прогноза.

Для проекта задержки рейса прогноз строится за 60 минут до планового отправления. В request допустимы `dep_hour` и `is_weekend`, но недопустимы фактическая задержка и любые признаки будущего. Это не только защита формата: API продолжает data contract ЛР2.

Попросите студентов разделить признаки на три группы: доступны клиенту; вычисляются сервером; запрещены из-за утечки. Такой разбор предотвращает ситуацию, когда удобство демонстрации незаметно меняет постановку ML-задачи.

## 3 Request model

Pydantic-модель задаёт типы, диапазоны, обязательность и политику дополнительных полей. Для учебного сервиса важна строгая семантика: строка `"9"` не должна молча становиться integer, а неизвестный ключ — игнорироваться.

```
class PredictionRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    dep_hour: StrictInt = Field(ge=0, le=23)
    is_weekend: StrictInt = Field(ge=0, le=1)
```

Объясните компромисс. Строгость раньше обнаруживает несовместимость, но требует дисциплины клиента. Допустимая нормализация тоже возможна, если она явная, документированная и покрыта тестом. Опасна не конверсия сама по себе, а случайная конверсия без договорённости.

Покажите четыре отрицательных примера: `dep_hour=24`, отсутствие `is_weekend`, строка вместо integer и лишнее поле `actual_delay`. Студент должен до запуска назвать ожидаемый status и элемент schema, который вызовет отказ.

## 4 Response model

Ответ также является контрактом. Если endpoint возвращает произвольный словарь, внутреннее поле может случайно стать публичным, а рефакторинг — сломать клиента.

Учебный response содержит вероятность, бинарное решение, threshold, версию модели и request ID. Важны инварианты:

- вероятность принадлежит `[0, 1]`;
- `delayed` вычисляется относительно возвращённого threshold;
- версия относится к фактически загруженному bundle;
- request ID совпадает с заголовком.

Версия приложения и версия модели — разные координаты. API `0.3.0` может обслуживать model bundle `0.2.0`; manifest или response должен позволять их связать.

## 5 Ошибка — тоже ответ

Необработанный stack trace не является API. Ошибка должна иметь стабильный код для программы, понятное сообщение и идентификатор для диагностики, но не раскрывать внутренние пути, секреты и персональные данные.

| Status | Смысл | Действие клиента |
|---|---|---|
| 200 | запрос выполнен | использовать результат |
| 404 | путь не существует | исправить URL или версию |
| 422 | request нарушает schema | исправить данные |
| 500 | необработанный дефект сервера | зарегистрировать инцидент |
| 503 | сервис жив, но зависимость не готова | повторить с backoff |

Подчеркните: ответ 200 с полем `error` разрушает машинную семантику. Полный текст сообщения Pydantic может меняться; стабильнее тестировать status, `error_code`, location и наличие деталей.

## 6 Health и readiness

`/healthz` отвечает на вопрос «процесс способен отвечать?». `/readyz` — «можно ли направлять сюда рабочие запросы?». Если модель отсутствует, процесс может быть жив, но не готов.

Смешение проверок создаёт две крайности: тяжёлый health вызывает перезапуски при временной проблеме зависимости; поверхностный readiness отправляет запросы в экземпляр без модели. Для ЛР7:

- `/healthz` не выполняет inference;
- `/readyz` проверяет загруженный bundle;
- `/predict` возвращает согласованный 503, если модель не готова.

## 7 Lifespan модели

Модель загружается один раз при старте приложения. Загрузка на каждый request увеличивает latency, создаёт файловую гонку и затрудняет диагностику. Lifespan задаёт переход состояния «инициализация → готов → завершение».

Различайте отсутствующий и повреждённый файл. Отсутствие модели может дать наблюдаемый режим not ready, если он предусмотрен эксплуатацией. Повреждённый bundle обычно должен остановить startup: принимать трафик с частично инициализированным состоянием опаснее явного отказа.

## 8 OpenAPI как исполняемая спецификация

FastAPI строит OpenAPI из route declarations и моделей. Это полезно только тогда, когда схема проверяется. Swagger UI — удобный клиент для исследования, но скриншот интерфейса не фиксирует весь контракт.

Предложите открыть `openapi.json` и найти:

1. `info.version`;
2. POST `/predict`;
3. ссылку на `PredictionRequest`;
4. обязательные поля и запрет дополнительных;
5. schemas ответов 200, 422 и 503;
6. `PredictionResponse` и `ErrorResponse`.

Схема сохраняется как build evidence и генерируется из принимаемого commit. Ручное редактирование JSON уничтожает прослеживаемость.

## 9 Совместимость

Сервер запускается — не значит, что изменение совместимо. Удаление поля, новое обязательное поле request, сужение диапазона и смена смысла существующего поля ломают часть клиентов.

Добавление необязательного поля ответа обычно совместимо, если клиенты игнорируют неизвестные ключи. Однако строгий клиент может упасть, поэтому фактические потребители важнее формальной таблицы.

Для несовместимого изменения нужны новая версия API, миграционное окно или параллельная поддержка. Версионирование без описания перехода лишь присваивает имя поломке.

## 10 Request ID

Request ID связывает клиентский симптом, response, лог и метрику. Middleware принимает безопасный идентификатор ограниченной длины или генерирует UUID и возвращает значение в header. Для JSON-ответов учебного API оно также включается в body.

Request ID не должен содержать email, token или payload. Он помогает корреляции, но не заменяет trace context. В распределённой системе дополнительно применяют стандартные trace/span identifiers.

Проведите демонстрацию: отправьте `X-Request-ID: lecture-case-1`, получите 422 и найдите то же значение в заголовке и теле. Затем повторите без header и покажите сгенерированный UUID.

## 11 Contract tests

Contract test проверяет наблюдаемое поведение границы. Минимальный набор включает:

- валидный request и полный response;
- граничные и ошибочные входы;
- not-ready mode;
- согласованность request ID;
- структуру OpenAPI;
- реальный HTTP-smoke внутри container.

`TestClient` даёт быстрый и детерминированный контур, но не обнаруживает ошибку команды запуска, порта или container network. Поэтому unit contract tests дополняются HTTP-smoke после Compose.

Избегайте хрупких проверок полного текста ошибок и порядка ключей JSON. Проверяйте семантические поля и инварианты.

## 12 Безопасность и эксплуатационные ограничения

Учебный сервис не включает полноценную аутентификацию, rate limiting и TLS termination. Это осознанное ограничение, а не рекомендация для production.

Даже в лаборатории необходимо:

- не логировать полный payload по умолчанию;
- не возвращать stack trace;
- ограничивать размеры и типы полей;
- не принимать признаки будущего;
- не включать секреты в image и OpenAPI examples;
- запускать container от non-root пользователя.

## 13 Демонстрационный сценарий

1. Запустить tests и экспортировать OpenAPI.
2. Запустить Compose и проверить `/healthz` и `/readyz`.
3. Выполнить валидный `/predict` с заданным request ID.
4. Передать строку вместо integer и объяснить 422.
5. Передать неизвестное поле и связать отказ с `additionalProperties: false`.
6. Запустить HTTP contract-smoke и открыть JSON-отчёт.
7. Временно убрать model bundle и сопоставить health=200, readiness=503.

Перед каждым действием аудитория прогнозирует результат. Это превращает показ команд в проверку ментальной модели.

## 14 Типичные ошибки

| Ошибка | Почему опасна | Как обнаружить |
|---|---|---|
| `dict` вместо request model | контракт не ограничен | schema и unknown-field test |
| неявное приведение типов | дефект клиента скрывается | wrong-type test |
| модель загружается на request | нестабильная latency | профилирование и review |
| health равен readiness | трафик идёт в неготовый pod | тест без модели |
| произвольный error JSON | клиент не может ветвиться | response-schema test |
| ручной OpenAPI | схема расходится с кодом | экспорт в CI |
| общий скриншот команды | нет индивидуального доказательства | личный negative case |

## 15 Вопросы аудитории

1. Почему дополнительное поле request может быть опасным, даже если код его игнорирует?
2. В каком случае 503 корректнее 500?
3. Почему `/docs` не заменяет сохранённый OpenAPI JSON?
4. Что произойдёт со строгим клиентом после добавления поля ответа?
5. Где должна загружаться модель и почему?
6. Какие данные нельзя включать в request ID?

## 16 Связь с ЛР7 и ЛР8

В ЛР7 команда реализует контракт, а каждый участник проводит собственный defect injection и защиту. В ЛР8 экспорт OpenAPI и contract tests становятся обязательными quality gates. Тем самым договор переходит из текста в правило слияния: несовместимое изменение не попадает в `main` без видимого красного check.

## 17 Итог

Надёжный inference API — не сетевой фасад над `predict`. Это проверяемый договор о доступных входах, ответах, ошибках, готовности, версиях и диагностике. Сильный контракт уменьшает число скрытых предположений между владельцами данных, модели, сервиса и клиента.

## 18 Официальные источники

- FastAPI request body: https://fastapi.tiangolo.com/tutorial/body/
- FastAPI response models: https://fastapi.tiangolo.com/tutorial/response-model/
- FastAPI error handling: https://fastapi.tiangolo.com/tutorial/handling-errors/
- FastAPI lifespan events: https://fastapi.tiangolo.com/advanced/events/
- FastAPI OpenAPI metadata: https://fastapi.tiangolo.com/tutorial/metadata/
- Pydantic models: https://docs.pydantic.dev/latest/concepts/models/
- OpenAPI Specification: https://spec.openapis.org/oas/latest.html